LIBRARY MANAGEMENT SYSTEM

A PROJECT REPORT

Submitted by

STUDENT NAME	STUDENT CODE	REGISTRATION NO.	ROLL NO.
AVIJAN MAITY	BWU/MCA/26/020	26102000138	26102410016

in partial fulfillment for the award of the degree

of

MASTER OF COMPUTER APPLICATIONS

IN

DEPARTMENT OF COMPUTATIONAL SCIENCES

BRAINWARE UNIVERSITY

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata - 700 125

ABSTRACT

Conventional library operations in educational institutions often rely on manual ledger books or disconnected spreadsheets, which are prone to data inconsistency, miscounted book copies, and error-prone manual calculation of overdue penalties. For small and medium-sized college libraries, a simple, reliable, and easily deployable tool is needed that does not require a database server or expensive software.

This project presents the **"LIBRARY MANAGEMENT SYSTEM,"** a terminal-based circulation and inventory management application built with **Python Object-Oriented Programming (OOP)** and backed by lightweight **JSON file storage**. The system models core library entities through the `Book` and `Student` classes, while the `Library` class manages data loading, saving, validation, and all circulation operations. The entire application is contained in a single `main.py` file and uses only the Python standard library.

Key features include: **(1) Validated Catalog Management** that checks author names using regular expressions and merges duplicate titles by increasing the copy count; **(2) Derived Availability Tracking** in which available copies are calculated from active issue transactions instead of being stored separately; **(3) Issue and Return Workflow** with date validation and an automatic overdue fine of ₹5 per day calculated at the time of return; and **(4) Search, History and Statistics** offering partial-match book search, a transaction history sorted by latest issue, and a summary of the most frequently issued titles.

The software was validated through manual functional testing of every menu option using valid, invalid, and boundary inputs, confirming correct input validation, accurate fine calculation, persistence of records across program restarts, and graceful handling of missing or corrupted data files.

Keywords: Library Management System, Object-Oriented Programming (OOP), JSON Data Storage, Book Circulation, Overdue Fine Calculation, Command-Line Interface (CLI), Input Validation.

TABLE OF CONTENTS

Chapter	Title	Page No.
	ABSTRACT	iii
	LIST OF TABLES	v
	LIST OF FIGURES	v
	LIST OF SYMBOLS & ABBREVIATIONS	vi
1.	**INTRODUCTION**	1
2.	**OBJECTIVES**	1
3.	**PLANNING & METHODOLOGY**	2
4.	**REQUIREMENT ANALYSIS**	2
5.	**SYSTEM FLOW & ARCHITECTURAL DIAGRAMS**	3
	5.1 System Context & Data Flow (CLD & DFD Level 0)	3
	5.2 Use Case & Entity–Relationship Models	3
	5.3 Circulation & Overdue Fine Lifecycle Flowchart	3
6.	**PROPOSED DESIGN & CLASS ARCHITECTURE**	4
7.	**SYSTEM MECHANISM, WORKFLOW & DATA HANDLING**	4
	7.1 End-to-End System Workflow	4
	7.2 Core Circulation Engine & Fine Calculation	5
	7.3 JSON Persistence & Derived Availability Tracking	5
8.	**CHALLENGES & PROBLEMS FACED DURING DEVELOPMENT**	5
9.	**EXPERIMENTAL RESULTS & TESTING**	6
10.	**FUTURE SCOPE & CONCLUSION**	6
	APPENDICES (A: Screens, B: Code, C: Data Files, D: Test Cases)	7
	REFERENCES	8

LIST OF TABLES

Table No.	Table Caption	Page No.
Table 1.1	Library Management System Operational Capabilities & Scope Overview	1
Table 3.1	Work Breakdown Structure (WBS) & Development Schedule	2
Table 4.1	System Functional Requirements Specification	2
Table 4.2	System Non-Functional Requirements & Performance Metrics	2
Table 7.1	End-to-End Circulation Workflow Stages & Internal Engine Actions	4
Table 9.1	Module-Wise Functional Testing Outcomes (Manual Tests)	6
Table C.1	JSON Data Structure Specification ('transactions.json' Records)	8
Table D.1	Representative Manual Test Cases Execution Matrix	8

LIST OF FIGURES

Figure No.	Figure Caption	Page No.
Figure 5.1	Context Level Diagram (CLD) of Library Management System	3
Figure 5.2	Data Flow Diagram Level 0 (DFD Level 0)	3
Figure 5.3	Unified System Use Case Diagram	3
Figure 5.4	Entity–Relationship Diagram (ERD) with Multiplicity Constraints	3
Figure 5.5	Complete Book Issue, Return & Overdue Fine Lifecycle Flowchart	3
Figure 6.1	Layered Single-File Architecture & Class Relationship Diagram	4
Figure 7.1	Stored vs. Derived Data Tracks & JSON Persistence Architecture	5
Figure A.1	Main Terminal Menu & Command Selection Screen Wireframe	7
Figure A.2	Return Confirmation & Overdue Fine Calculation Output Wireframe	7

LIST OF SYMBOLS, ABBREVIATIONS & NOMENCLATURE

Symbol / Abbreviation	Meaning / Definition
OOP	Object-Oriented Programming (Encapsulation, Classes, Objects, Modular Separation)
CLI	Command-Line Interface (Terminal-driven interactive menu)
JSON	JavaScript Object Notation (Lightweight text format used to store records on disk)
ISO 8601	International date format YYYY-MM-DD used for issue, due, and return dates
Regex	Regular Expression (Pattern matching used to validate author names)
PK / FK	Primary Key / Foreign Key (Identifier fields that link records across entities)
CRUD	Create, Read, Update, Delete (Basic data record operations)
DFD	Data Flow Diagram (Graphical depiction of data movement through a system)
ERD	Entity–Relationship Diagram (Conceptual data model representation)
WBS	Work Breakdown Structure (Hierarchical task decomposition in planning)
ASCII	American Standard Code for Information Interchange (Text-based UI tables)
FINE_PER_DAY	Program constant that defines the overdue fine per day (₹5)
I/O	Input / Output (Console input and reading / writing of data files)
ID	Auto-generated identifier (B / S / T prefix followed by a yymmddHHMMSS timestamp)

CHAPTER 1
INTRODUCTION

Academic libraries form the intellectual nucleus of higher education institutions, serving hundreds of students and faculty members daily. In many college departments, circulation operations—such as catalog tracking, book checkouts, returns, and overdue fee collections—are still conducted through manual paper ledgers or obsolete monolithic systems. These legacy workflows suffer from severe record discrepancies, untracked inventory depletion, and delayed financial settlement of library dues.

The **LIBRARY MANAGEMENT SYSTEM** is engineered as a lightweight, terminal-driven automation tool built using **Python Object-Oriented Programming (OOP)** and **JSON file storage**. Operating in a continuous interactive menu loop, the application integrates catalog management, student registration, book issue and return, overdue fine calculation, searching, and library statistics into a single, zero-dependency program.

Table 1.1: Library Management System Operational Capabilities & Scope Overview

Module	Feature Scope	Underlying Entity	Module	Feature Scope	Underlying Entity
`Catalog`	Add books, merge duplicates, track copies	`Book`	`Fines`	Overdue days × ₹5 calculated at return	`Fine Field`
`Members`	Register students, unique email validation	`Student`	`History`	Transactions sorted by latest issue	`Transaction`
`Circulation`	Issue with due date, return by transaction ID	`Transaction`	`Analytics`	Totals, availability, top issued titles	`Statistics`

** The due date is entered by the librarian at the time of issue (it cannot be in the past); the overdue fine accrues at ₹5/day.*

The system emphasizes **data reliability and simple operation**: every input is validated before it is accepted, unique IDs are generated automatically and displayed to the user, all lookups are case-insensitive, and records are written to the `data/` folder immediately after every change, so no information is lost between sessions.

CHAPTER 2
OBJECTIVES

The project was designed to satisfy key educational and operational objectives, bridging clean software design principles with real-world practical utility:

2.1 Primary Objective

To design, implement, and test a terminal-based library management program in Python using Object-Oriented Programming, loops, conditional branching, regular expressions, date handling, JSON file persistence, and exception handling.

2.2 Functional & Technical Objectives

• **OOP Modeling:** Construct encapsulated domain classes (`Book`, `Student`, and `Library`) with a clear separation between data records, business rules, and console interaction.
• **Reliable Data Persistence:** Store books, students, and transactions in JSON files that are loaded at startup and saved after every change.
• **Accurate Availability Tracking:** Calculate available copies from active issue records so that stock counts can never drift out of sync.
• **Automated Fine Calculation:** Compute overdue days on return and charge ₹5/day (`FINE_PER_DAY`), recording the fine in the transaction.
• **Input Validation & Fault Tolerance:** Validate names, emails, quantities, and dates, and handle missing or corrupted data files without crashing.

CHAPTER 3
PLANNING & METHODOLOGY

The software was built using an **iterative incremental development lifecycle**, prioritizing modular component design, early working versions of each feature, and continuous manual verification after every increment.

3.1 Work Breakdown Structure (WBS)

WBS Code	Project Phase	Core Deliverables	Timeline
1.0	Domain Modeling & Data Files	Entity classes (Book, Student), JSON data files, load and save routines	Week 1–2
2.0	Library Service & Circulation	Library class, issue and return methods, available-copies calculation	Week 3–4
3.0	Overdue Fine Calculation	Date arithmetic, FINE_PER_DAY rule, fine stored in the transaction record	Week 5–6
4.0	Console Menu & Displays	Aligned ASCII tables, search, 11-option main menu loop	Week 7–8
5.0	Validation & Error Handling	Regex author check, date and quantity parsing, safe file loading	Week 9–10
6.0	Testing & Documentation	Manual functional testing, edge cases, project report	Week 11–12

3.2 Risk Mitigation & Quality Planning

Risk 1: Inventory Mismatch: Mitigated by never storing an available-copies counter; `available_copies()` derives it from the number of active `Issued` transactions and issuing is blocked when fewer than one copy remains.
Risk 2: Missing or Corrupted Data Files: Solved through `load_data()`, which creates missing files, catches `JSONDecodeError` and `OSError`, and starts with an empty list instead of crashing.
Risk 3: Invalid User Input: Addressed by validating every input (regular expression, integer and date parsing) and wrapping each menu action in a `try-except` block inside `main()`.

CHAPTER 4
REQUIREMENT ANALYSIS

System requirements were categorized into functional operations and non-functional engineering standards:

Table 4.1: System Functional Requirements Specification

Req. ID	Feature Module	Functional Description
FR-01	Catalog Management	Add books with a positive number of copies, merge duplicate title–author entries, and display all or only available books.
FR-02	Member Registration	Register students with a non-blank name and a unique, valid-looking email; list all registered students.
FR-03	Book Issue	Validate book and student IDs (case-insensitive), check that a copy is available, accept a due date that is not in the past, and create an `Issued` transaction.
FR-04	Return & Overdue Fine	Return a book by transaction ID, count the days late against the due date, and record a fine of ₹5 per overdue day.
FR-05	Transaction History	List all issue records, newest first, with due date, status, and fine amount.
FR-06	Search & Display	Search by ID, title, or author (partial, case-insensitive) and show total and available copies in aligned tables.
FR-07	Analytics & Statistics	Compute title count, total and available copies, currently issued copies, registered students, and the top five most issued books.

Table 4.2: System Non-Functional Requirements & Performance Metrics

Attribute	Target Specification	Implementation Mechanism
Data Persistence	Records survive program restarts	JSON files in `data/` written by `save_all()` after every change
Zero Dependencies	Pure Python 3 standard library	Uses built-in `json`, `os`, `re`, and `datetime`
Fault Tolerance	No unhandled crash on bad input or bad files	Input validation, safe `load_data()`, and `try-except` in the menu loop
Usability	Simple, self-explanatory interaction	Numbered 11-option menu, aligned tables, and clear success / error messages

CHAPTER 5
SYSTEM FLOW & ARCHITECTURAL DIAGRAMS

The architectural models below illustrate the data flow, interactions, data structures, and execution lifecycles of the Library Management System:

5.1 System Context & Data Flow (CLD & DFD Level 0)

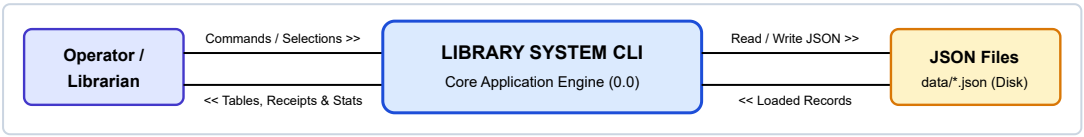


Figure 5.1: Context Level Diagram (CLD) of Library System

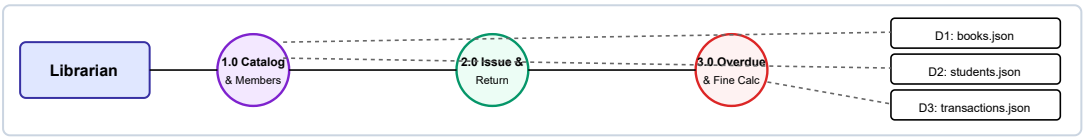


Figure 5.2: Data Flow Diagram Level 0 (DFD Level 0)

5.2 Use Case & Entity–Relationship Models

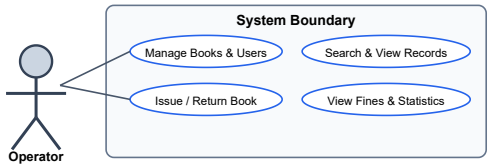


Figure 5.3: Unified System Use Case Diagram

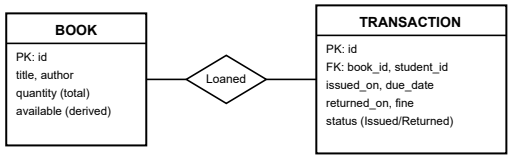


Figure 5.4: Entity–Relationship Diagram (ERD)

5.3 Circulation & Overdue Fine Lifecycle Flowchart

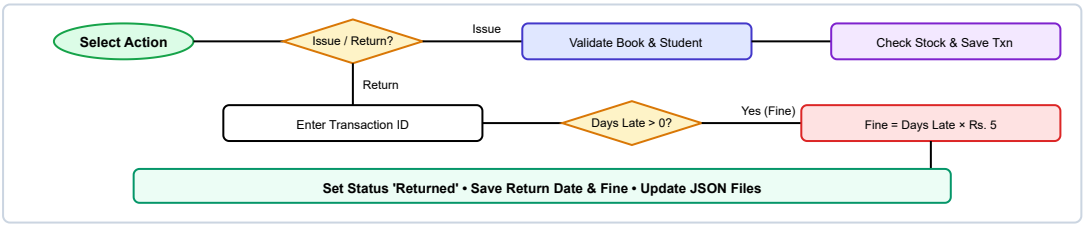


Figure 5.5: Complete Book Issue, Return & Overdue Fine Lifecycle Flowchart

CHAPTER 6
PROPOSED DESIGN & CLASS ARCHITECTURE

The software is organized into clear logical layers inside the single file `main.py`, keeping presentation, business rules, domain entities, and data storage separate.

6.1 Layered Single-File OOP Architecture

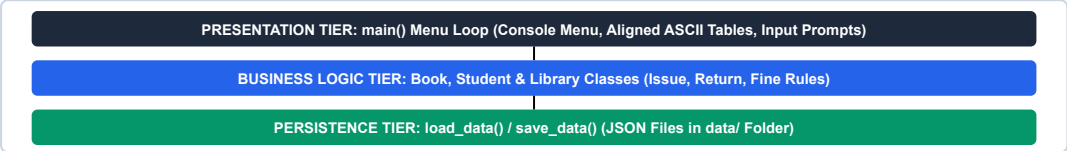


Figure 6.1: Layered Single-File Architecture & Class Relationship Diagram

6.2 Console Interface & Input Validation

Every interaction takes place through a numbered menu of eleven options (0–10). Book listings are shown in aligned columns for *ID*, *Title*, *Author*, *Total*, and *Available* copies, so the librarian can read the required book ID directly from the screen before issuing. Student IDs appear in the student list, and transaction IDs are displayed after every issue and in the transaction history. All ID lookups are case-insensitive, and every prompt reports a clear message when the input is invalid.

CHAPTER 7
SYSTEM MECHANISM, WORKFLOW & DATA HANDLING

7.1 End-to-End System Workflow (How the Application Works)

The continuous execution cycle within `main.py` operates through eight sequential stages:

Step	Workflow Phase	Underlying System Operations & Actions
1	**Initialization**	`main()` creates the `Library` object; its constructor creates the `data/` folder and loads the books, students, and transactions JSON files (creating empty files if missing).
2	**Menu Dispatching**	Displays 11 choices (0–10). The input is stripped and mapped to `Library` methods through an action dictionary; invalid choices show a message.
3	**Inventory Ingestion**	Option [1] prompts for Title, Author (letters and spaces only), and Copies (≥ 1); a matching title and author increases the quantity, otherwise a new book with ID `B` + timestamp is saved.
4	**Member Registration**	Option [5] registers students with a non-blank name and a unique email containing "@", storing them with ID `S` + timestamp.
5	**Book Issue**	Option [7] validates book and student IDs and availability (copies ≥ 1), reads a due date (YYYY-MM-DD, not in the past), and appends an `Issued` transaction with ID `T` + timestamp.
6	**Return & Overdue Fine**	Option [8] takes the transaction ID, calculates overdue days against the due date, sets the fine (₹5/day), and marks the transaction `Returned`.
7	**Display & Search**	Options [2]–[4], [6], and [9] show all books, available books, search results, registered students, and the transaction history (newest first).
8	**Statistics & Exit**	Option [10] reports totals, available and issued copies, student count, and the top five issued titles; option [0] saves all records and exits.

7.2 Core Circulation Engine & Fine Calculation

The circulation engine executes simple, deterministic logic. When a transaction is returned, the `Library.return_book()` method compares the recorded due date with the actual return date:

```
Overdue Days = max(0, (Return Date - Due Date).days)
Fine = Overdue Days × FINE_PER_DAY (₹5)
Status = "Returned"
```

The calculated fine, the return date, and the new status are written into the transaction record, and the overdue days and fine amount are displayed on screen. Because available copies are computed as `quantity − issued transactions`, marking a transaction as `Returned` automatically makes the copy available again.

7.3 JSON Persistence & Derived Availability Tracking

The system keeps a clear separation between data that is stored on disk and values that are computed on demand:



Figure 7.1: Stored vs. Derived Data Tracks & JSON Persistence Architecture

CHAPTER 8
CHALLENGES & PROBLEMS FACED DURING DEVELOPMENT

Engineering a dependable terminal software environment required resolving several key technical challenges:

8.1 Keeping Book Availability Consistent

Problem: Storing an “available copies” counter beside the total quantity could drift out of sync with the actual issue records after edits or interrupted runs.
Solution: Availability is never stored; `available_copies()` calculates it as the total quantity minus the number of `Issued` transactions for that book.

8.2 Overdue Calculation Boundary Edge Cases

Problem: Returning a book early or exactly on the due date produces a zero or negative date difference, which must never create a fine.
Solution: Dates are stored in ISO format and converted with `date.fromisoformat()`; the overdue days are clamped using `max(0, ...)` so only late returns are charged.

8.3 Missing or Corrupted Data Files

Problem: On the first run the JSON files do not exist, and a damaged file would normally crash the program with a decoding error.
Solution: `load_data()` creates empty files when they are missing, catches `JSONDecodeError` and `OSError`, prints a warning, and continues with an empty list.

8.4 Unique IDs and Input Validation

Problem: Without a database, unique IDs had to be generated by the program, and incorrect inputs (such as numbers in author names or invalid dates) could corrupt records.
Solution: IDs are built from a prefix and a timestamp (`B`, `S`, `T` + `yymmddHHMMSS`), author names are checked with a regular expression, and dates and quantities are parsed inside `try-except` blocks.

CHAPTER 9
EXPERIMENTAL RESULTS & TESTING

The application was validated through **manual functional testing** of every menu option, using valid, invalid, and boundary inputs across 7 focused test areas:

Table 9.1: Module-Wise Functional Testing Outcomes (Manual Tests)

Test Area	Test Scenarios Covered	Expected Result	Status
Book Management	Valid add, blank title / author, non-letter author, zero or non-numeric copies, duplicate title merge.	Correct record created or clear error shown; duplicate increases quantity.	**PASS**
Student Registration	Valid registration, email without "@", duplicate email in a different letter case.	Invalid and duplicate entries rejected.	**PASS**
Book Issue	Valid issue, wrong book / student ID, no copies left, past due date, wrong date format.	Issue blocked with a message; valid issue creates a transaction.	**PASS**
Return & Fine	On-time return, late return, unknown transaction ID, repeated return.	Fine = days late × ₹5; repeated return rejected.	**PASS**
Search & Display	Partial ID / title / author search, no match, available-only listing.	Correct matches; "No matching books found." when empty.	**PASS**
Data Persistence	Restart after changes, missing data files, corrupted JSON file.	Records reloaded; empty list and warning on a bad file.	**PASS**
Statistics	Empty system, and after several issues and returns.	Correct totals and top-five ranking.	**PASS**

9.2 Observations & Performance

• **Responsiveness:** All operations are simple in-memory list scans followed by a small JSON write, so no noticeable delay is expected at the scale of a college department library.
• **Data Safety:** Records are saved after every change and on exit, so completed operations are not lost between sessions.
• **Footprint & Portability:** Zero external dependencies; the whole program is a single `main.py` file that runs on any system with Python 3.

CHAPTER 10
FUTURE SCOPE & CONCLUSION

10.1 Future Scope

The modular OOP design provides a good foundation for future extensions:
1. **Database Backend:** Moving from JSON files to SQLite or MySQL for faster queries, safer concurrent access, and sequential or UUID-based identifiers.
2. **Fine Payment & Reminders:** Tracking whether a fine has been paid and sending automatic overdue reminder emails to students.
3. **Graphical / Web Interface & Testing:** A Tkinter or Flask front end, barcode-based book lookup, and an automated unit test suite.

10.2 Conclusion

The **LIBRARY MANAGEMENT SYSTEM** demonstrates that a command-line application can deliver reliable library automation without heavyweight dependencies. By combining Object-Oriented design, JSON-based persistence, derived availability tracking, strict input validation, and an automatic overdue fine calculation, the project provides a practical and easy-to-use solution for managing books, students, and circulation records in an academic library, and offers a solid base for future enhancement.

APPENDIX A: USER INTERFACE SCREENS & TERMINAL WIREFRAMES

The wireframes below depict actual interactive command-line interface layouts and user interaction flows:

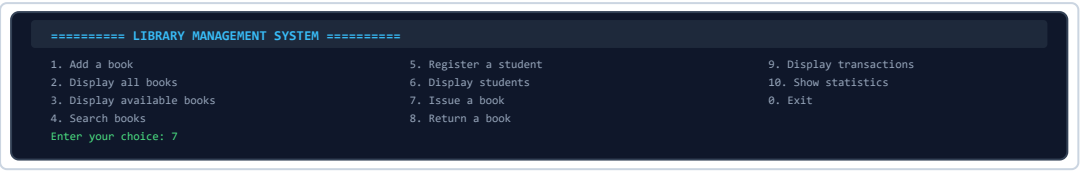


Figure A.1: Interactive CLI Main Menu & Command Selection Layout



Figure A.2: Return Confirmation & Overdue Fine Calculation Output

APPENDIX B: SAMPLE CODE & CORE IMPLEMENTATION

Code B.1: Return Book & Overdue Fine Logic (main.py)

```
def return_book(self):
    transaction_id = input("Enter transaction ID: ").strip()
    transaction = next((item for item in self.transactions
                        if item["id"].lower() == transaction_id.lower()), None)
    if transaction is None:
        print("Error: Invalid transaction ID.")
        return
    if transaction["status"] != "Issued":
        print("This book has already been returned.")
        return
    returned_date = date.today()
    due_date = date.fromisoformat(transaction["due_date"])
    overdue_days = max(0, (returned_date - due_date).days)
    fine = overdue_days * FINE_PER_DAY
    transaction["returned_on"] = returned_date.isoformat()
    transaction["fine"] = fine
    transaction["status"] = "Returned"
    self.save_all()
```

Code B.2: Available Copies Calculation (main.py)

```
def available_copies(self, book_id):
    book = self.find_book(book_id)
    if book is None:
        return 0
    issued_count = sum(1 for transaction in self.transactions
                       if transaction["book_id"] == book_id
                       and transaction["status"] == "Issued")
    return max(0, book["quantity"] - issued_count)
```

APPENDIX C: DATA FILES & DATA DICTIONARY

Table C.1: JSON Data Structure Specification ('transactions.json' Records)

Field Name	Data Type	Constraint / Format	Description & Business Rules
`id`	String	Unique; “T” + yymmddHHMMSS	Unique transaction identifier generated at the time of issue.
`book_id`	String	References books.json `id`	Identifies the borrowed book.
`student_id`	String	References students.json `id`	Identifies the borrowing student.
`book_title`, `student_name`	String	Copied at issue time	Stored in the record so the history can be shown without extra lookups.
`issued_on`	String	ISO YYYY-MM-DD	Date on which the book was issued (system date).
`due_date`	String	ISO YYYY-MM-DD; not in the past	Return deadline entered by the librarian.
`returned_on`	String / null	ISO date or null	Actual return date (null while the book is still issued).
`fine`	Integer	Default 0	Overdue fine in rupees (days late × ₹5), set on return.
`status`	String	“Issued” or “Returned”	Current state; “Issued” records reduce the available copies.

APPENDIX D: TEST CASES & REFERENCES

Table D.1: Representative Manual Test Cases Execution Matrix

Test ID	Test Scenario	Test Condition & Inputs	Expected Result	Result
TC-01	Stock Depletion Guard	Issue a book when all copies are already issued	Issue rejected	**PASS**
TC-02	Duplicate Email	Register a student with an existing email (different letter case)	Registration rejected	**PASS**
TC-03	Past Due Date	Issue a book with a due date earlier than today	Date rejected	**PASS**
TC-04	Overdue Fine Accrual	Return a book 3 days after its due date	Fine = Rs. 15	**PASS**
TC-05	Repeated Return	Return the same transaction ID twice	Second return rejected	**PASS**

REFERENCES

1. Python Software Foundation (2026) *The Python Standard Library: json — JSON encoder and decoder*, docs.python.org.
2. Lutz, M. (2013) *Learning Python: Powerful Object-Oriented Programming*, 5th edn, O'Reilly Media.
3. Pressman, R. S. and Maxim, B. R. (2020) *Software Engineering: A Practitioner’s Approach*, 9th edn, McGraw-Hill Education.
4. Python Software Foundation (2026) *The Python Standard Library: datetime — Basic date and time types*, docs.python.org.
5. Brainware University (2026) *Guidelines for MCA Academic Project Documentation and Viva Voce*, Department of Computational Sciences.